

DMML_Short_Notes

I. Data Management Fundamentals

A. Core Concepts

- **Definition of Data Management:** The process of transforming data into a format and storage organization suitable for later stages of the ML process, including training, anonymization, and deletion.
- **Purpose in ML Environments:** To prepare data for ML models, ensure data quality, manage sensitive information, and comply with regulations.
- **Key Operations:** Training ML models, anonymizing sensitive data, deleting data, and preparing for future exploration.
- **Business Input:** Collaborating with business leaders to understand primary use cases and explore future data utilization.

B. Phases of Data Management

1. Creation:

- **Origin of ML Training Data:** Datasets are generated or captured through various processes (e.g., system logs, images from events, diagnostic data).
- **Static vs. Dynamic Datasets:** Some datasets are effective when static, while others require frequent updates to remain relevant.
- **Types of Data: Structured Data:** Quantitative, predefined schema, highly organized (tabular formats like spreadsheets, relational databases), easily processed by simple code.
- **Unstructured Data:** Qualitative, no standard schema, difficult to process with conventional tools (e.g., text, images, audio).
- **Semi-structured Data:** No specific schema but includes tags and semantic markers (e.g., JSON, XML), lies between structured and unstructured.
- **Dataset Augmentation:** Techniques (e.g., Snorkel) to expand small datasets into larger ones, yielding good results at low cost, but requiring caution.

1. Ingestion:

- **Process:** Receiving data into the system and writing it to storage for further processing.

- **Filtering and Selection:** Not all created data is ingested; a necessary step to select relevant data.
- **Sampling:** Reducing data volume to save computational costs, but may impact quality. Important to measure quality cost vs. savings.
- **Mechanism:** Can range from simple file dumps to sophisticated RPC endpoints for tracking data bundles.
- **Reliability Concerns:** Correctness (data properly read and written) and throughput. Monitoring data existence and condition is critical.

1. **Processing:**

- **Validation:** Ensuring data quality, especially important for external sources or when a schema exists. Storing and referencing standard definitions.
- **Cleaning and Ensuring Data Consistency:** Addressing missing fields, duplicates, misclassifications, and encoding errors. Normalization (transforming data to a similar scale) and bucketing (mapping data into smaller groups).
- **Enriching and Extending:** Combining data with data from other sources.
- **Labeling:** Identifying events/records with outside confirmation (often human). Crucial for supervised ML, can be challenging and expensive.
- **Joins:** Using external data sources to extend training data.

1. **Post-processing:**

- **Storage:** How and where data is stored, driven by use cases (training and serving). Predominant concerns: efficiency (driven by access patterns, model/team structure) and metadata (for human interaction and understanding model construction).
- **Management:** Motivated by business purpose (problem solving, value) and model structure/strategy (what models, how built, refresh frequency, similarity).
- **Analysis and Visualization:** Transforming large datasets into easy-to-navigate representations using statistical/graphical techniques. Providing explanations of data meaning, links between datasets, and data quality. Requires high-performant visualization tools.

II. Data and ML Pipelines

A. ML Pipelines vs. Conventional Pipelines

- **Purpose:** Extract usable and repeatable insights from data, consuming data and outputting processed representations.
- **Key Differences:** ML pipelines have specific constraints, success is hard to measure, failures are difficult to detect, and they are highly dependent on underlying data systems.
- **Focus:** Data used to construct models and the processing pipeline that transforms data into models.

B. Data as an Asset and Liability

- **Asset:** Essential for ML systems, more/higher-quality data often outperforms sophisticated models with less data. Organizations scramble to collect data for value (e.g., Netflix recommendations).
- **Liability:Complexity:** Acquisition, collection, and curation reveal unexpected nuance, leading to potential harm.
- **Compliance:** Must adhere to laws (location, origin), organizational policies, industry practices, insurance regulations, corporate governance. Restrictions on PII, consent, access, and deletion.
- **Security:** Data must be secured from external/internal access. Pseudonymization (reversible replacement of identifiers) and permanent removal of direct connections to private data.
- **Deletion:** Data deletion is surprisingly hard due to multiple copies and metadata management. Important to facilitate user requests for deletion.

C. Data Sensitivity and Reliability

- **Sensitivity:** ML pipelines are unusually sensitive to subtle changes in data distribution; small omissions can lead to significant errors (e.g., losing data from a specific region).
- **Reliability Characteristics:Durability:** Data is not lost.
- **Consistency:** Data is the same across all copies.
- **Version Control:** Changes are tracked over time.
- **Performance:** Data is read fast enough.
- **Availability:** Data is ready to be read.

III. Data Management Framework and Components

A. Data Management Discipline

- **Definition:** Ingesting, processing, securing, and storing an organization's data for strategic decision-making.
- **Challenges:** Big data growth leads to complexity, data silos, security risks, and bottlenecks.
- **Solutions:** Cleaning, unifying, and securing data to glean insights and empower data science.

B. Framework Components (Trellance/DAMA DMBOK)

- **Data Governance:** Overarching support through stewardship, policies, processes, standards, and adherence to best practices.
- **Data Architecture:** Infrastructure for storage, integration, and use of data.
- **Metadata:** Critical information about data attributes for efficient use.
- **Data Quality:** Structure for data to fulfill business needs.
- **Data Lifecycle:** Integrity from initial introduction to final deletion.
- **Analytics:** Statistical and visualization techniques for insights.
- **Data Privacy:** Supporting internal and external data sharing.

C. Data Management Strategy Components

- **Data Processing:** Ingesting raw data (web APIs, IoT, etc.), then processing via ETL/ELT (filtering, merging, aggregating) for specific purposes (BI dashboards, ML algorithms).
- **Data Storage:** Dictated by data type and purpose.
- **Data Warehousing:** Defined schema, structured data (relational), for specific analytics requirements (dashboards, BI).
- **Data Lakes:** Incorporates relational and non-relational data (structured/unstructured), a sandbox for innovative projects (benefiting data scientists).
- **Data Governance:** Standards and processes for effective data asset leverage (quality, access, usability, security). Data governance councils align taxonomies and define roles for privacy.
- **Data Security:** Guardrails against unauthorized access, corruption, or theft. Leveraging encryption and data masking.

D. Data Platform Infrastructure

- **Requirements:** Robust data and computing infrastructure that scales to meet demand without performance impact.
- **Common Builds:** Data lakes (raw data, Hadoop/cloud), data warehouses (performance reporting on structured data), temporary sandboxes.
- **Environment Needs:** Fast query processing, large storage, elastic computing, in-memory caches, columnar storage, large in-memory data grids/computing clusters.

IV. Data Sources

A. Characteristics and Access Patterns

- ML systems work with diverse data sources, each with different characteristics, access patterns, purposes, and processing methods.
- Understanding sources helps efficient data access and manipulation.

B. Common Data Sources

- **User Input Data:Definition:** Explicitly input by users (texts, images, videos, files).
- **Challenges:** Easily mal-formatted, requires heavy-duty checking and processing. Users expect immediate results, so fast processing is needed.
- **System-Generated Data:Definition:** Data generated by system components (logs, model predictions).
- **Logs:** Record system state, events (memory, services). Less likely to be mal-formatted, can be processed periodically, but volume can grow quickly. Signal loss in noise is a problem; specialized services (Logstash, DataDog) help. Storage strategies include low-access storage.
- **System-Generated User Data:** Users' behaviors (clicks, scrolls). Subject to privacy regulations. Used for ML predictions and training.
- **Internal Databases:** Generated by services/enterprise applications (inventory, CRM, users). Used directly by models or ML system components (e.g., Amazon search queries).
- **Third-Party Data:First-party:** Data a company collects on its own users/customers.
- **Second-party:** Data collected by another company on its customers, made available to you.

- **Third-party:** Data collected by companies on the general public who aren't their customers.
- **Challenges:** Privacy concerns (anonymization is hoped for). Usually sold as structured, cleaned data.

V. Data Formats

A. Need for Serialization

- **Persistence:** Data needs to be stored ("persisted") after being acquired.
- **Considerations:** Storing multimodal data, cheap and fast access, storing complex models, human readability, access patterns, text vs. binary.
- **Data Serialization:** Converting data structures/objects into a format for storage/transmission and later reconstruction (encoding/decoding).

B. Common Data Formats

- **JSON (JavaScript Object Notation): Characteristics:** Language-independent, human-readable, key-value pair paradigm, handles various structuredness levels.
- **Example:** Structured vs. unstructured JSON blobs.
- **CSV (Comma-Separated Values) and Parquet: Row-major (CSV):** Consecutive elements in a row stored contiguously. Faster for row-based access and data writes. Better for frequent writes.
- **Column-major (Parquet):** Consecutive elements in a column stored contiguously. Faster for flexible column-based reads, especially with large data. Better for frequent column-based reads.
- **Text vs. Binary Formats: Text (CSV, JSON):** Plain text, human-readable.
- **Binary (Parquet):** Contains 0s and 1s, meant for programs to interpret. More compact (e.g., Parquet is faster to unload, consumes less storage).

C. Structured vs. Unstructured Data (Revisited)

- **Structured Data:** Predefined schema (e.g., age: number, name: string). Easier analysis. Disadvantage: commitment to schema (updates, multiple sources). Stored in data warehouses.
- **Unstructured Data:** No predefined schema. Flexible storage, can store any type (converted to byte strings). Stored in data lakes (often raw data before processing).

- **Key Differences:** Schema definition, ease of search/analysis, arrival speed, schema change impact, storage location.

VI. Data Models

A. Importance and Characteristics

- **Impact:** Profoundly affects problem-solving and software writing.
- **Description:** How data is represented in terms of attributes (e.g., car: make, model, year, color).
- **Assumptions:** Every data model embodies assumptions about its intended use.
- **Selection Impact:** Affects system building and solvable problems.

B. Relational Model

- **Origin:** Invented by Edgar Codd (1970). Still popular.
- **Structure:** Data organized into relations (tables), each row is a tuple. Relations are unordered. Usually stored in CSV or Parquet.
- **Normalization:** Desirable to reduce redundancy and improve data integrity (e.g., 1NF, 2NF). Downside: expensive joins for large tables.
- **Relational Databases:** Built around this model.
- **Querying (SQL):** Declarative language (specifies output, not steps). Database optimizer decides execution.

C. NoSQL (Not Only SQL)

- **Motivation:** Relational model restrictions (strict schema, painful schema management).
- **Origin:** Hashtag for non-relational database discussions; reinterpreted as "Not Only SQL."
- **Types:Document Model:Concept:** Self-contained "documents" (JSON, XML, BSON) with unique keys. Collections analogous to tables, documents to rows.
- **Flexibility:** Documents in a collection can have different schemas.
- **Locality:** All information for an item in one place, easier to read. Difficult to execute joins across documents.
- **Schema-less Misconception:** Applications still assume some structure. Shifts responsibility from writing to reading application.

- **Graph Model:Concept:** Graphs of nodes and edges (relationships). Focus on relationships between data items.
- **Retrieval:** Faster to retrieve data based on relationships.
- **Combined Use:** Common to use both document and relational models for different tasks (e.g., PostgreSQL, MySQL support both).

VII. Query Languages for Data

A. Imperative vs. Declarative Queries

- **Imperative Code:** Instructs computer on *how* to perform operations in a specific order (line by line, conditions, variable updates). Hard to parallelize.
- **Declarative Queries (SQL, Relational Algebra):** Specifies *what* data is needed (pattern, conditions, transformations), not *how* to achieve it. Database query optimizer decides execution. Lends itself to parallel execution.

B. MapReduce Querying

- **Programming Model:** For processing large amounts of data in bulk across many machines (popularized by Google).
- **Nature:** Limited form, supported by some NoSQL stores (MongoDB, CouchDB). Neither fully declarative nor fully imperative, but in between.
- **Logic:** Expressed with "map" and "reduce" code snippets.
- **Restrictions:** Map and reduce functions must be pure functions (no side-effects, use only input data). Allows parallel execution and re-runs on failure.
- **Complexity:** Low-level, requires writing two coordinated functions. Higher-level languages (SQL) can be implemented via MapReduce.
- **Aggregation Pipeline (MongoDB):** Declarative query language similar to SQL (JSON-based syntax).

VIII. Formats for Encoding Data

A. In-Memory vs. Persistent/Network Data

- **In-Memory:** Data in objects, structs, lists, optimized for CPU access.
- **Disks/Networks:** Data encoded as self-contained byte sequences (e.g., JSON). Requires translation (encoding/serialization/marshalling) and reverse

(decoding/parsing/deserialization/unmarshalling).

B. Language-Specific Formats

- **Examples:** Java's `java.io.Serializable`, Ruby's `Marshal`, Python's `pickle`.
- **Convenience:** Easy to save/restore in-memory objects.
- **Problems: Integration:** Tied to one language, difficult to read in others.
- **Security:** Decoding can instantiate arbitrary classes, posing risks (e.g., remote code execution).
- **Versioning:** Often an afterthought, neglecting forward/backward compatibility.
- **Efficiency:** CPU time and size of encoded structure often neglected (e.g., Java's bad performance).
- **Conclusion:** Bad for anything other than very transient purposes.

C. Standardized Encodings

- **Examples:** XML, JSON, CSV.
- **Characteristics:** Written/read by many languages, textual, somewhat human-readable.
- **Criticisms:** XML verbose/complicated; JSON popular for browser support/simplicity; CSV less powerful. Widely used and disliked.
- **Subtle Problems: Numbers:** Ambiguity (number vs. digit string in XML/CSV), JSON doesn't distinguish integers/floats or specify precision (large numbers can be inaccurate). Workarounds needed.
- **Binary Strings:** JSON/XML good for Unicode text, but not binary strings. Base64 encoding used as a workaround.
- **Optional Schema Support:** XML/JSON schemas are optional, powerful but complex. Many tools don't use them. CSV has no schema. Applications need to handle changes manually.
- **Utility:** Good for many purposes, especially data interchange between organizations, as long as formats are agreed upon.

IX. Modes of Data Flow

A. Data Flow Through Databases

- **Process:** Writer encodes, reader decodes.

- **Compatibility: Backward Compatibility:** Necessary for a single process accessing its own older data.
- **Forward Compatibility:** Often required when multiple processes (newer and older versions) access the database concurrently.
- **Data Outlives Code:** Database contents persist much longer than application code.
- **Schema Evolution:** Possible to change schema without rewriting all existing data (e.g., adding null-default columns). Allows database to appear consistent despite mixed schema versions.
- **Archival Storage:** Data dumps often encoded using the latest schema, good opportunity for analytics-friendly formats (e.g., Parquet).

B. Data Flow Through Services (Client and Server)

- **Clients/Servers:** Servers expose API, clients make requests.
- **Web Clients:** Browsers request HTML, CSS, etc. using standardized protocols (HTTP, HTML). Other clients (mobile apps, Ajax) make HTTP requests for data (e.g., JSON).
- **Service-Oriented Architecture (SOA)/Microservices:** Servers can be clients to other services. Goal: easier change/maintenance via independent deployability. Requires compatibility across service API versions (old and new running concurrently).
- **Web Services (REST and SOAP):** **REST:** Design philosophy (not protocol) based on HTTP principles. Simple data formats, URLs for resources. Popular for cross-organizational integration and microservices.
- **SOAP:** XML-based protocol, often over HTTP. Complex web service framework (WS-*), WSDL for API description. Relies heavily on tooling. Less popular now.
- **Remote Procedure Calls (RPC):** Attempts to make remote calls look like local function calls. Fundamentally flawed due to differences. Newer RPC frameworks (Thrift, Avro, gRPC) are more explicit, provide service discovery, and can offer better performance with binary encoding. REST is predominant for public APIs; RPC for inter-service communication within an organization.

C. Message Passing Data Flow

- **Nature:** Between RPC and databases. Client sends message to message broker (stores temporarily), then delivered to recipient. Asynchronous.
- **Message Broker Advantages:** Buffers if recipient unavailable/overloaded (reliability).

- Redelivers messages on crash (prevents loss).
- Sender doesn't need recipient's address.
- One message to multiple recipients.
- Logically decouples sender/recipient.
- **Communication:** Usually one-way, sender doesn't wait for reply.
- **Landscape:** Open source (RabbitMQ, Kafka) popular.
- **Mechanism:** Process sends message to named queue/topic; broker delivers to consumers/subscribers. Many producers/consumers per topic.
- **Data Model:** Brokers typically don't enforce specific data models (message is bytes). Backward/forward compatible encoding provides flexibility.

X. Data Architecture

A. Definition and Need

- **Definition:** Design of systems to support evolving data needs, through flexible and reversible decisions evaluating trade-offs.
- **Need:** Rapidly changing data requirements, new tools. Understanding good data architecture is crucial for successful data engineering.
- **Aspects:Operational Architecture:** Functional requirements (people, processes, technology). *What* needs to happen.
- **Technical Architecture:** *How* data is ingested, stored, transformed, and served.

B. Traditional Architectures

- **Data Warehouse (DW):Definition:** Central data hub for reporting/analysis. Highly formatted, structured data. Subject-oriented, integrated, non-volatile, time-variant.
- **Evolution:** Historically expensive, now accessible with cloud (pay-as-you-go).
- **Organizational Architecture (ETL):** DW pulls data from applications. Extract (source), Transform (clean, standardize, business logic), Load (DW target). Data loaded into data marts.
- **ELT DW Architecture:** Data moved directly to staging in DW. Transformations handled by DW system (batches, transformed output to tables/views). Popularized during big data (Hadoop).

- **Cloud DW:** Separation of compute from storage (object storage for limitless storage). On-demand compute power. Blurring line with data lakes. (e.g., Redshift, BigQuery, Snowflake).
- **Data Marts:** Refined subset of DW, focused on single sub-organization. Improves performance for complex queries.
- **Data Lake:Definition:** Dump all data (structured/unstructured) into a central location.
- **Data Lake 1.0 (HDFS/Object Storage):** Limitless/cheap storage. Query/transform with unlimited compute (MapReduce, Spark).
- **Shortcomings (Data Swamps):** Unmanageable sizes, poor schema management, cataloging, discovery. Write-only (updates/deletes difficult). Processing challenges (joins).
- **Data Lakehouse:Definition:** Convergence of DW and data lakes (Databricks). Incorporates DW controls, data management, structures, while housing data in object storage. Supports ACID transactions.
- **Technical Similarities:** Cloud DWs and data lakehouses share features (separate compute/storage, petabyte queries, variety of objects, integration with Spark/Beam).
- **Future:** Vendors combine capabilities (AWS, Azure, Google Cloud, Snowflake, Databricks).

C. Emerging Architectures

- **Modern Data Stack (MDS):Aim:** Reduce complexity, increase modularization. Cloud-based, plug-and-play, off-the-shelf components.
- **Components:** ELT data pipeline, cloud-based columnar warehouse/data lake, data transformation, data management/governance, monitoring, visualization, exploration.
- **Characteristics:** Easy setup, pay-as-you-go, plug-and-play modularity (open standards/APIs).
- **Key Building Blocks:Data Ingestion:** Batch/streaming (Fivetran, Hevo Data, Singer, Airflow).
- **Data Storage & Processing:** DW (BigQuery, Redshift, Snowflake), data lakes (S3, ADLS Gen2, GCS with Spark/Athena), data lakehouses (combining features).
- **Data Transformation:** dbt (SQL), Airflow + Python/R/SQL.
- **Business Intelligence & Analytics:** Self-service platforms (Looker, Mode, Tableau).

- **Data Cataloging & Governance:** Discovery, trust, context (Atlan, Apache Atlas, DataHub).
- **Data Privacy & Access Governance:** Managing privacy controls/access across stack (Immuta, Okera, Apache Ranger).
- **Other Tools:** Real-time processing (Kafka, Druid), data science (Jupyter, Sagemaker), event collectors (Segment), data quality (Great Expectations).
- **Lambda Architecture (Batch + Streaming):Context:** Early 2010s, reconciling batch and streaming data.
- **Components:** Batch layer (data warehouse, precomputed views), speed layer (stream processing, low latency), serving layer (combines views).
- **Shortcomings:** Managing multiple systems with different codebases is difficult, error-prone, hard to reconcile.
- **Kappa Architecture (Stream-First):Response to Lambda:** Use stream processing as backbone for all data handling (ingestion, storage, serving). Real-time and batch seamlessly applied to same data.
- **Shortcomings:** Not widely adopted. Streaming is complex/expensive; batch processing is more efficient for historical data.
- **Dataflow Model (Unified Batch/Stream):Inspiration:** From Lambda/Kappa's challenges. Google's Dataflow model (Apache Beam).
- **Core Idea:** All data as events, aggregation over windows (unbounded for streams, bounded for batches). Batch as a special case of streaming.
- **Adoption:** Frameworks like Flink and Spark adopted similar approaches.
- **Data Mesh:Response to Monolithic Platforms:** Decentralization, applying domain-driven design to data architecture. Domain-oriented data ownership, data as a product, self-serve data infrastructure, federated computational governance.

XI. Big Data Applications and Systems

A. Big Data Architecture Style

- **Definition:** Designed for data too large or complex for traditional systems.
- **Workloads:** Batch processing, real-time processing, interactive exploration, predictive analytics/ML.

- **When to Use:** Store/process large volumes, transform unstructured data, capture/process unbounded streams.

B. Components of Big Data Systems

- **Data Sources:** Databases, files, IoT devices.
- **Data Storage:** Distributed file stores for high volumes of large files.
- **Batch Processing:** Long-running jobs for filtering, aggregating, preparing data for analysis.
- **Real-time Message Ingestion:** Capture/store real-time messages for stream processing.
- **Stream Processing:** Filter, aggregate, prepare real-time data for analysis.
- **Analytical Data Store:** Structured format for querying (data warehouse, etc.).
- **Analysis and Reporting:** Provide insights.
- **Orchestration:** Automate workflows (Azure Data Factory, Apache Oozie/Sqoop).

C. Benefits and Challenges

- **Benefits:** Technology choices, performance through parallelism, elastic scale, interoperability.
- **Challenges:** Complexity (numerous components), skillset (specialized technologies), technology maturity (evolving tools).

D. TDS (Traditional Data Stack) Setup (Problems)

- **Long Turn-around Time:** Untangling/setting up on-premise infrastructure is costly (engineers, interconnections).
- **Slow Response to New Information:** Scaling is costly/time-consuming. Data pipelines take hours, insights delayed.
- **Expensive Journey to Insights:** Manual report generation, cleaning, transfer leads to errors, wasted time, inability to scale.

XII. Data Pipeline

A. Definition and Purpose

- **Definition:** Series of steps to move data from one system to become useful in another (analytics, data science, ML/AI).

- **Working:** Pulls data from source, applies transformations/processing, pushes to destination.
- **Purpose:** Transform raw data for analytics, applications, ML/AI. Deliver sales data, link scientists, recommend services, track cases, predictive maintenance.

B. Benefits and Challenges

- **Benefits:** Self-service data (ad hoc creation, faster innovation), accelerate cloud migration (new use cases, cost-effectiveness, NLP/sentiment/image processing), real-time analytics/applications (continuous data delivery).
- **Challenges: Always Under Construction:** Building/debugging takes time, business requirements change, backlogs.
- **Out of Order Data Pipelines:** Small changes require massive rework (data drift). Pipelines go offline for updates.
- **Build It and They Will Come:** Tied to specific frameworks/processors. Changing infrastructure means rebuilding.

C. Components and Architectures

- **Working Components:** Data sources (applications, messaging, databases, cloud storage, DW, DL), transformations (masking PII, type conversion, calculations, renaming, joining, schema generation, CDC), destinations.
- **Architectures: Batch Data Pipeline:** Moves large sets at specific times/thresholds (bulk ingestion, ETL).
- **Streaming Data Pipeline:** Continuous data flow (populating data lakes, real-time applications, fraud detection).
- **Change Data Capture (CDC) Pipeline:** Refreshes data, syncs systems by sharing only changes. Useful for cloud migration.

D. Data Pipeline Tools

- **Scaling Challenge:** From single pipeline to thousands.
- **Considerations:** Data transformation needs, environment stability, pipeline operationalization.
- **Tool Categories: Data Ingestion/Loading Tools:** Easy setup, basic transformations. Solve "under construction," but rigid (data drift requires rebuilds).

- **Data Integration/Transformation Platforms (ETL):** Complex solutions for every scenario. Designed for static data, break with change.
- **Data Engineering Platforms (DataOps):** Build "smart data pipelines" (abstract "how," focus on "what"). Resilient to change, adaptable to new platforms. Design/deploy in hours, not weeks.

E. Data Pipeline People (Data Analytics Pipelines)

- **Immaturity:** Most organizations lack mature processes to convert data to insights. Handcrafted, non-repeatable, isolated work.
- **Results:** Slow development, error-prone operations, business-IT conflict.
- **Origin:** Emerged from big data. Data lakes provided access to raw data, creating diverse workflows (clean, filter, aggregate, move, load).
- **Complexity:** Simple (export CSV) to complex (multiple sources, merge, dimensional schema, aggregate, flag nulls, BI extracts, dashboards).
- **Types:Development Pipeline:** Creates code to process data.
- **Execution Pipeline:** Runs pipeline in production.
- **Inefficiencies:** Often managed by different teams, increasing errors and delays.
- **Generic Pipeline Stages:** Data Ingestion, Data Transformation, Data Analysis, Data Visualization.
- **Specialists & Tools:** Data engineers (ETL/data prep), BI developers (visualization), data scientists (Python/R, predictive models).

XIII. DataOps

A. Definition and Foundations

- **Need:** Increasing complexity of analytics pipelines, growing teams, need for better collaboration/governance.
- **Definition:** Emerging practices, processes, technologies for building/enhancing data/analytics pipelines to meet business needs quickly.
- **Goal:** Increase agility/cycle times, reduce defects, increase confidence in analytics output.
- **Foundations:** Builds on Agile, Lean, CI/CD (software engineering), but adapted for data/analytics environments (multiple sources, varied use cases). Relies on test

automation, code repositories, collaboration tools, orchestration, workflow automation.

B. Benefits and Pitfalls

- **Benefits:** Improve collaboration/communication (blur lines, knowledge exchange), accelerate time to production (streamlined, automated pipelines), increase quality/reliability (automated tests, monitoring).
- **Common Pitfalls:****Acceptance:** Don't make it technical; show business value.
- **Engineering:** Keep it simple; balance effort/benefit; avoid over-complication.

XIV. Key Components of Modern Data Infrastructure

A. Diversity of Data Sources

- Most organizations have dozens/hundreds of internal and third-party data sources.
- Example: E-commerce combining shopping cart data (PostgreSQL) with web analytics (Google Analytics).
- Data pipelines start with ingestion from multiple sources.

B. Ingestion Interface and Data Structure

- **Interface:** Databases (Postgres, MySQL), REST API, stream processing (Kafka), network file systems/cloud storage (logs, CSVs), DW/DL, HDFS/HBase.
- **Structure:** JSON (REST API, flat files), structured (MySQL), semi-structured logs, CSV/FWF.
- Each presents unique challenges/opportunities.

C. Cloud Data Warehouses and Data Lakes (Revisited)

- **Transformative Factors:** Public cloud providers (ease of deployment, dropping storage costs), highly scalable columnar databases (Redshift, Snowflake, BigQuery).
- **Data Warehouse:** Structured, optimized for reporting/analysis.
- **Data Lake:** Unstructured, high volume, variety of data types.
- Both have a place, data pipelines often move data between them.

D. Data Ingestion Tools

- **Common Tools:** Singer, Stitch, Fivetran.

- **Custom Code:** Reasons: cost, build-over-buy culture, security/legal concerns with vendors.
- **Focus:** Extract and load steps (some tools offer simple transforms). Goal: ease ingestion for data engineers and analysts.

E. Data Transformation and Modeling Tools

- **Data Transformation:** Broad term (T in ETL/ELT). Simple (timestamp conversion, PII masking) to complex (metrics from multiple columns, aggregation, filtering).
- **Data Modeling:** Specific transformation. Structures/defines data for analysis, optimized for it. Usually one or more tables in a data warehouse.
- **Tools:** Simple transforms often in ingestion tools. Complex transformations/modeling use specialized tools (dbt, Airflow + Python/R/SQL). SQL is accessible and common.
- **Recommendation:** Frameworks supporting SQL for data models preferred over point-and-click.

F. Workflow Orchestration Platforms

- **Need:** Manage scheduling and flow of tasks in complex pipelines (dozens/hundreds of steps, Python ingestions, SQL transformations).
- **Tools:** Apache Airflow, Luigi, AWS Glue (general); Kubeflow Pipelines (specific for ML/Docker).

G. Customizing Data Infrastructure

- Rare to find identical infrastructures. Organizations pick/choose tools, build the rest.
- Build vs. Buy decision depends on culture, resources, constraints (dollars, engineers, security, legal risk).
- Understanding trade-offs is crucial for high-quality infrastructure.

XV. Lifecycle of a Data Science Project

A. Paradigm Shift

- ML/AI/Data Science becoming radically more approachable, enabling more smarter applications.

B. Common Elements

- **Variation:** Lifecycle details vary between projects (e.g., predictive model for customer lifetime value vs. self-driving cars).
- **Universals:** Data and computation are foundational. Integration into production systems requires software engineering. Experimentation and iteration are central.
- **Benefits of Common Infrastructure:** Increase project volume, speed time-to-market, ensure robustness, support variety of projects.

XVI. Data Science Infrastructure (The Stack)

A. Evolution

- From primordial ML/optimization libraries to a "Cambrian explosion" of data science libraries, frameworks, infrastructures (often commercial).
- Widely shared patterns will form open-source infrastructure.

B. Components of the Stack (Bottom to Top)

1. **Data Warehouse:** Storage, critical for discoverability, accessibility, durability. Must not lose data.
2. **Compute Resources:** Apply algorithms to data. Needs on-demand resources, often in parallel. Executes algorithms blindly.
3. **Job Scheduler:** Manages repeating processes (ML model training). Feeds fresh data regularly. Must be highly available, manage concurrent machines/pipelines.
4. **Architecture:** Defines application (algorithms, how scheduling connects algorithms/data pipelines). Software engineering challenge: mapping problems to robust architecture.
5. **Versioning:** Essential for experimentation/iteration. Architectures change. Run multiple versions side-by-side (isolated algorithms, data, pipelines).
6. **Model Operations:** Deploy, monitor, assure validity of models. Tracks metadata about executions/models from prototype to production.
7. **Feature Engineering:** Sequence of data transformations from raw input. Major part of data scientist's work. Challenge: make efficient.
8. **Model Development:** Defining mathematical model to transform features into outputs. Data scientist's expertise: choosing/implementing modeling approach. Small part of end-to-end machinery.

C. Why the Stack? (Common Themes)

- **Foundational (Data Warehouse, Compute):** Generic, agnostic to what's executed. Data and compute are universal.
- **Middle Layers (Job Scheduler, Architecture, Versioning, Model Operations):** Define software architecture, integrate existing components.
- **Top Layers (Feature Engineering, Model Development):** Realm of data science, evolving quickly with experimentation.

D. Data Scientists: Freedom and Responsibility

XVII. Data Serving

A. Three Ways to Serve Data

1. **Serve Data for Analytics and BI:** Traditional area. Prepare data for statistical analysis, reporting, dashboards. Stakeholders need visibility.
2. **Serve Data for ML Applications:** ML needs high-quality, prepared data. Work with data scientists/ML engineers for acquisition, transformation, delivery. Data engineers may handle ML-specific tasks (featurization).
3. **Serve Data Through Reverse ETL:** Sending data back to data sources (e.g., adtech platform bids, lead scoring to CRM). Increasingly recognized as a formal data engineering responsibility.

B. Ways to Serve Data for Analytics and ML

1. **File Exchange:** Ubiquitous. Process data, generate files (text, CSV, images) for consumers. Depends on use case, consumer processes, file size/number, access, data type.
2. **Databases (OLAP):** Critical layer. Querying and consuming results. Imposes order/structure (schema), fine-grained permissions, high performance for intensive queries. Data scientists extract data for feature engineering and model training.
3. **Streaming Systems:** Increasingly important for serving emitted metrics (different from traditional queries). Operational analytics databases combine OLAP with stream processing.
4. **Query Federation:** Pulls data from multiple sources (DL, RDBMS, DW). Distributed query virtualization engines (Trino, Presto, Starburst) gain recognition. Allows ad hoc queries, blending data without pipelines. Read-only access to source systems. Good for access/compliance.

5. **Serving Data in Notebooks:** Data scientists use notebooks (Jupyter/JupyterLab) for exploration, feature engineering, model training. Programmatically connect to data sources (API, database, DW, DL) using libraries.

XVIII. Three Levels of ML Software (Workflow)

A. Overview

- Goal: Build statistical model using data and ML algorithms.
- Challenges: Building successful ML-based software is difficult.
- Three Main Assets: Data, Model, Code.
- Essential Technical Methodologies: Data Engineering, ML Model Engineering, Code Engineering.

B. Data Engineering

- **Initial Step:** Acquire and prepare data (integration from various resources/formats).
- **Data Preparation:** Iterative/agile process for exploring, combining, cleaning, transforming raw data into curated datasets. Most expensive/time-consuming. Aims to avoid error propagation.
- **Sequence of Operations:**
 - Data Ingestion:** Collect data (Spark, HDFS, CSV, synthetic data, enrichment). Maximize automation.
 - Data Exploration:** Data profiling (content/structure). Output: metadata (max, min, avg).
 - Data Validation:** User-defined error detection functions to spot errors. Assess data quality.
 - Data Wrangling (Cleaning):** Re-formatting, correcting errors (missing values, duplicates, encoding). Normalization. Programmatic scripts recommended for reuse.
 - Data Labeling:** Assigning specific categories to data points (human or automated). Key driver for supervised ML.
 - Data Splitting:** Training, validation, and test datasets.

C. Model Engineering

- **Phase:** Writing and executing ML algorithms to obtain an ML model.
- **Operations:**
 - Model Training:** Applying ML algorithm on training data. Includes feature engineering, hyperparameter tuning.

- **Model Evaluation:** Validating trained model against objectives.
- **Model Testing:** Final "Model Acceptance Test" with hold-back test dataset.
- **Model Packaging:** Exporting final model to a specific format (PMML, PFA, ONNX) for consumption by business application.

D. Model Deployment (Code Engineering)

- **Purpose:** Deploy model as part of business application (mobile, desktop). ML models need feature vectors for predictions.
- **Operations: Model Serving:** Addressing the ML model artifact in a production environment.
- **Model Performance Monitoring:** Observing performance with live data (prediction deviation). Signals can trigger retraining.
- **Model Performance Logging:** Every inference request results in a log-record.

XIX. Machine Learning Lifecycle

A. Overview

- Cyclic iterative process with instructions/best practices across defined phases. Adds clarity/structure.
- Phases are not strictly sequential, can have feedback loops.
- **Phases:** Business goal identification, ML problem framing, Data processing, Model development, Model deployment, Model monitoring.

B. Detailed Phases

1. **Business Goal Identification:** Clear problem, business value, measurable objectives. Involve stakeholders.
2. **ML Problem Framing:** Frame business problem as ML problem (what to observe, predict). Define success criteria, performance metrics.
3. **Data Processing:** Convert data to usable format.
 - **Data Collection:** Identify, evaluate means for collecting data.
 - Labeling (manual/automated).
 - Ingesting/Aggregating from multiple sources.

- Data technologies: storage (SQL, data lakes, DW), ETL, data lake.
 - **Data Preparation:** Data preprocessing, feature engineering.
 - EDA with visualization tools.
 - Data Preprocessing: Right shape/quality for training. Cleaning (outliers, duplicates, missing values), partitioning (train/validate/test sets to prevent overfitting, data leakage), scaling (normalize/standardize), unbiasing/balancing, augmenting (synthesizing new data).
 - Feature Engineering: Select/transform variables. Creation (one-hot, binning, splitting), transformation/imputation (Cartesian products, non-linear), extraction (dimensionality reduction like PCA), selection (subset of relevant features).
1. **Model Development:** Select algorithm, train model.
 - **Model Building, Training, Tuning:** Feature selection (after bias strategy).
 - Building code (version control, CI/CD).
 - Algorithm selection (experiments, parameter tuning, accuracy, explainability, time, memory).
 - Model training (distributed training, model/data parallelism).
 - Debugging/profiling (system bottlenecks, overfitting, gradients).
 - Validation metrics (loss, accuracy).
 - Hyperparameter tuning (learning rate, epochs, hidden layers).
 - Training code container (container images for quick/reliable deployment).
 - Model artifacts (trained parameters, definition, metadata).
 - Visualization.
 - **Model Evaluation:** Assess performance/accuracy (multiple models, error rates). Offline (holdout set) or online (live data). Fine-tune data/algorithm.
 1. **Deployment:** Deploy trained, tuned, evaluated model to production. Make predictions/inferences.
 - **Deployment Strategies:** Blue/green, canary, A/B, shadow deployment/testing to reduce downtime/risks.
 - **Components:** Inference pipeline (automates capturing prepared data, predictions, post-processing), Scheduler pipeline (re-training at intervals to minimize drift).

1. **Monitoring:** Capture data, compare to training set, detect issues, send alerts.
 - **Issues:** Data quality, model quality, bias drift, feature attribution drift.
 - **Components:** Model explainability (evaluate soundness), detect drift (data/concept drift, performance degradation), model update pipeline (if alarm manager identifies violations).

C. ML Lifecycle Architecture Diagram (Components)

- **Online/Offline Feature Store:** Reduces duplication of feature engineering. Online for real-time inference (low latency), offline for training/batch scoring (history).
- **Model Registry:** Repository for ML model artifacts (trained model, metadata). Enables lineage, version control.
- **Performance Feedback Loop:** Automates model performance evaluation.
- **Model Drift Feedback Loop:** Automates model update/re-training tasks.
- **Alarm Manager:** Receives alerts from monitoring, publishes notifications.
- **Scheduler:** Initiates re-training at defined intervals.
- **Lineage Tracker:** Enables reproducible ML, recreates environment at specific point-in-time.

XX. Machine Learning Workflow: Data Engineering Pipelines (Detailed)

A. Importance of Data

- Fundamental part of ML workflow. Good datasets hugely impact model quality/performance ("Garbage In, Garbage Out").
- Data engineering is time-consuming (majority of project time).
- Goal: Create training and testing datasets.

B. Stages

- **Data Ingestion:Process:** Collecting data from various systems/formats (databases, data marts, Spark, HDFS, synthetic data). Maximally automated.
- **Actions:** Data Sources Identification (provenance), Space Estimation/Location, Obtaining Data (convert format), Back up Data (work on copy), Privacy Compliance (anonymize), Metadata Catalog, Test Data (sample and put aside to avoid snooping bias).

- **Exploration and Validation:Exploration (Profiling):** Obtain content/structure info. Output: metadata (max, min, avg).
- **Validation:** User-defined error detection functions. Assess data quality.
- **Actions:** Use RAD tools (Jupyter), Attribute Profiling (name, records, type, numerical measures, missing values, distribution), Label Attribute Identification, Data Visualization, Attributes Correlation, Identify Additional Data.
- **Data Wrangling (Cleaning):Process:** Programmatically wrangle data (re-formatting, restructuring). Scripts/functions for reuse.
- **Actions:** Identify Transformations, Fix/Remove Outliers, Fill Missing Values (zero, mean, median), Drop Not Relevant Data, Restructure Data (reordering, creating new fields, combining, filtering, shifting granularity).
- **Data Splitting:**Splitting data into training, validation, and test datasets for ML stages.

XXI. Machine Learning Workflow: Model Engineering Pipelines (Detailed)

A. Core of ML Workflow

- Writing and executing ML algorithms to obtain an ML model. Utilized by data science team.
- Operations: Model Training, Model Evaluation, Model Testing, Model Packaging. Automation recommended.

B. Model Training

- **Process:** Applying ML algorithm on training data. Includes feature engineering, hyperparameter tuning.
- **Feature Engineering (Specifics):** Discretize continuous, decompose (categorical, date/time), add transformations (log, sqrt, x^2), aggregate, scale (standardize/normalize). New features added quickly.
- **Iterative Workflow:** Code review/versioning for model specification. Train many models (linear, logistic, k-means, etc.) with standard parameters. Measure/compare performance (N-fold cross-validation). Error Analysis. Further feature selection/engineering. Identify top models. Hyperparameter tuning. Consider Ensemble methods.

C. Model Evaluation, Testing, and Packaging

- **Model Evaluation:** Validate trained model meets business objectives before production.
- **Model Testing:** Measure performance with hold-back test dataset (final "Model Acceptance Test").
- **Model Packaging:** Exporting final ML model into specific format (PMML, PFA, ONNX) for business application.

D. ML Model Serialization Formats

- **Purpose:** Distributable format, executable as independent asset (outside training environment).
- **Flavors: Language-agnostic: Amalgamation:** Simplest. Bundle model and necessary code as one package (single source file). (e.g., SKompiler for Scikit-learn to SQL, Excel, PFA, SymPy expressions).
- **PMML:** XML-based for model serving. Not all ML algorithms, licensing issues for open source.
- **PFA (Portable Format for Analytics):** JSON-formatted text, describes scoring engine. Designed as PMML replacement.
- **ONNX (Open Neural Network eXchange):** ML framework independent. Supported by big tech. Consumed by ONNX-enabled runtimes.
- **Vendor-specific:** Scikit-Learn (.pkl), H2O (POJO/MOJO), SparkML (MLEap), TensorFlow (.pb), PyTorch (.pt), Keras (.h5), Apple (.mlmodel for Core ML).

XXII. Different Forms of ML Workflows (Architectural Styles)

A. Classification Dimensions

- ML Model Training (Offline learning, Online learning)
- ML Model Prediction (Batch predictions, Real-time predictions)

B. Training Patterns

- **Offline Learning (Batch/Static):** Model trained on already collected data. Remains constant until retrained. "Model decay" possible.

- **Online Learning (Dynamic):** Model regularly retrained as new data arrives (streams). For time-series data. Incremental learning.

C. Prediction Patterns

- **Batch Predictions:** Deployed model makes predictions based on historical input data. For non-time-dependent data or non-critical real-time.
- **Real-time Predictions (On-demand):** Predictions generated in real-time using available input data.

D. ML Architecture Patterns

1. **Forecast:** Widely spread in academia/education. Easiest ML system to create. Not very useful for industry production systems. Trains model on dataset, runs on historical data, makes predictions.
2. **Web-Service (Microservices):** Most common deployment. Takes input, outputs prediction. Model trained offline, uses real-live data for predictions. Model constant until retrained/redeployed. Runs near real-time, single record.
3. **Online Learning (Real-time Streaming Analytics):** Model continuously receives data stream, incrementally retrained. Instantly available as web service. Works with Lambda architecture. Difficulty: bad data can degrade performance.
4. **AutoML:** Training ML models with minimal effort/expertise. User provides data, system selects/configures algorithm. Experimental, usually by big cloud providers. Executes entire training pipeline in production for new models.

XXIII. Machine Learning Workflow: Code: Deployment Pipelines (Detailed)

A. Final Stage of ML Project

- Model Serving, Model Performance Monitoring, Model Performance Logging.

B. Model Serving Patterns

- **Components:** Inference (getting data to compute predictions), automated retraining/deployment pipeline, prediction API.
1. **Five Patterns:Model-as-Service:** Wrap ML model as independent service (web service, REST API, gRPC). Used for Forecast, Web Service, Online Learning.

2. **Model-as-Dependency:** Straightforward. Packaged ML model as dependency within software application (e.g., Jar). Mostly Forecast.
3. **Precompute:** Train model, precompute predictions for batch data. Persist predictions in database. Query database for results. Tightly related to Forecast.
4. **Model-on-Demand:** ML model as runtime dependency (separate release cycle). Message-broker architecture typical. Broker component (event channels/message queues), Event processor (model serving runtime, ML model).
5. **Hybrid-Serving (Federated Learning):** Multiple models (server-side general, user-side specialized). Devices train local models, send updates to server. Server model adjusted. Benefits: privacy (personal data stays on devices), accurate models. Constraints: less powerful devices, distributed training data.

C. Deployment Strategies

- Wrapping trained models as deployable services.
1. **Docker Containers to Cloud Instances:** Containerization (Docker) is de-facto standard. Package ML tech stack/code in Docker container. Orchestration by Kubernetes/AWS Fargate. ML functionality via REST API (Flask app).
 2. **Serverless Functions:** Cloud vendors provide ML platforms (SageMaker, GCP AI, Azure ML). Package application code/dependencies in .zip files (single entry point function). Managed by Azure Functions, AWS Lambda, Google Cloud Functions. Constraints: artifact size.

D. Data Science and ML in Real World

- **Gaining Importance:** Core capabilities for complex problems, transforming industries.
- **Ingredients for Effective ML:** Large datasets, inexpensive on-demand compute, specialized accelerators, rapid ML research advances.
- **Real Challenge:** Not just building ML model, but building an integrated ML system and continuously operating it in production. MLOps (unifying ML system Dev/Ops).

E. Elements of ML System

- Small fraction is ML code. Rest is vast/complex: configuration, automation, data collection/verification, testing/debugging, resource management, model analysis, process/metadata management, serving infrastructure, monitoring.
- MLOps applies DevOps principles to ML systems.

F. DevOps vs. MLOps

- **DevOps Benefits:** Shortening dev cycles, increasing deployment velocity, dependable releases. Concepts: CI (merge working copies frequently), CD (produce software in short cycles, reliably release any time).
- **ML Systems Differences: Team Skills:** Data scientists/ML researchers (exploratory, model development) vs. software engineers.
- **Development:** Experimental nature (different features, algorithms). Tracking what worked, reproducibility.
- **Testing:** More involved (data validation, model quality/validation, unit/integration tests).
- **Deployment:** Multi-step pipeline for automatic retraining/deployment.
- **Production:** Model performance decay (data profiles, degradation). Monitoring (summary stats, online performance).
- **ML Differences (Process): CI:** Tests/validates code, components, *data*, *data schemas*, *models*.
- **CD:** Not single software package, but *system (ML training pipeline) that deploys another service (model prediction service)*.
- **CT (Continuous Training):** New property unique to ML. Automatically retraining and serving models.

XXIV. Maturity of the ML Process (MLOps Levels)

A. Defining Maturity

- Level of automation defines maturity (velocity of training new models).
- Three levels:
 1. MLOps level 0: Manual process
 2. MLOps level 1: ML pipeline automation
 3. MLOps level 2: CI/CD pipeline automation

B. MLOps Level 0: Manual Process

- **Characteristics:** Manual, script-driven, interactive (notebooks).
- Disconnection between ML and operations (data scientists hand over artifacts). Can lead to training-serving skew.

- Infrequent release iterations (few, rarely changing models).
- No CI (testing in notebooks).
- No CD (infrequent deployments).
- Deployment refers to prediction service.
- Lack of active performance monitoring.
- **Challenges:** Models break in real world. Need active monitoring, frequent retraining, continuous experimentation with new implementations.

C. MLOps Level 1: ML Pipeline Automation

- **Aim:** Continuous training by automating ML pipeline.
- **Characteristics:** Rapid experiment (orchestrated, automated transitions).
- CT of model in production (automatically trained on fresh data via live triggers).
- Experimental-operational symmetry (same pipeline implementation in dev/pre-prod/prod).
- Continuous delivery of models (pipeline delivers prediction services).
- Modularized code for components/pipelines (reusable, composable, containerized).
- Pipeline deployment (entire training pipeline deployed, not just model).
- Automated data validation (schema/value skews, anomalies) and model validation (predictive quality, comparison to current model, consistency, infrastructure compatibility).
- Feature store (centralized repository for definition, storage, access of features). API for batch/real-time serving. Avoids training-serving skew.
- Metadata management (records pipeline execution info: versions, times, executor, parameters, artifacts, previous model pointers, evaluation metrics).
- ML pipeline triggers (on demand, schedule, new training data, performance degradation, concept drift).
- **Challenges:** Suitable for new models based on new data, not new ML ideas. Manual testing of pipeline.

D. MLOps Level 2: CI/CD Pipeline Automation

- **Aim:** Full CI/CD automation of ML pipeline.

- **Stages:** Development and experimentation (iterative ML algorithms, modeling, push to source repo).
- Pipeline continuous integration (build, test code/components, package artifacts). Includes unit testing feature engineering, model methods, preventing NaN values, component integration.
- Pipeline continuous delivery (deploy artifacts to target env, deployed pipeline with new model implementation). Verifying compatibility with infrastructure, testing prediction service (API calls, performance/load testing), validating data, automated/semi-automated/manual deployment.
- Automated triggering (pipeline executed based on schedule/trigger, trained model to model registry).
- Model continuous delivery (serve trained model as prediction service).
- Monitoring (collect stats on model performance, trigger new experiment/pipeline). Data analysis/model analysis still manual.
- **Summary:** Implements ML in production as an automated ML pipeline, uses CI/CD for new pipeline implementations. Allows coping with rapid changes. Gradual implementation is possible.

XXV. Distributed and Parallel Computing for Big Data

A. Need for High Data Processing

- **Parallel Computing:** Additional resources on same system, complex computations divided into subtasks, processed in parallel. Increases processing capability.
- **Distributed Computing:** Multiple resources connected in network, tasks distributed. Increases speed/efficiency. Suitable for huge data in limited time.

B. Techniques for High Data Processing

- **Cluster/Grid Computing:** Multiple servers share workload. Used in Hadoop.
- **Massively Parallel Processing (MPP):** Single machine acts as grid (storage, memory, computing). Used in data warehouses (Greenplum, ParAccel).
- **High-Performance Computing (HPC):** High performance/scalability with IMC. Processes floating-point data at high speeds. Specialty/custom applications.

C. Comparing Distributed and Parallel Systems

- **Distributed:** Independent, autonomous systems in network. Loose coupling. Each has own memory/CPU.
- **Parallel:** Computer system with several processing units. Common shared memory. Tight coupling.

D. Data Models

- **Distributed Databases:** Tables/relations, schema, data partitioning.
- **Hadoop:** Flat files (any format), no schema, automatic block division.

E. Computing Models

- **Distributed Databases:** Transactions, ACID properties, distributed transactions.
- **Hadoop:** Jobs divided into tasks (Map/Reduce). MapReduce computing model.

XXVI. Introduction to Hadoop

A. Big Data Challenges

- Volume (terabytes of data)
- Variety (structured, semi-structured, unstructured)
- Velocity (fast generation speed)

B. Why Hadoop?

- Handles massive volume, variety, velocity.
- **Advantages:** Low cost (open source), computing power (many nodes), scalability (easy to add), storage flexibility (unstructured data), inherent data protection (replication factor).

C. What is Hadoop?

1. **Two Tasks:Massive Data Storage:** Huge amounts across nodes using low-cost commodity storage. Solves storage challenge by Replication Factor (RF).
2. **Faster Data Processing:** Application development, parallel computation on nodes. Solves processing challenge by MapReduce Programming.

D. Hadoop Ecosystem and Architecture

- Image Source: Edureka, Intellipaat (refer to diagrams in source).

XXVII. In-Memory Computing for Big Data

A. Hadoop Issues

- Apache Hadoop revolutionized big data processing (low cost, batch applications).
- Reliance on persistent storage (fault tolerance) and one-pass computation (MapReduce) makes it poor for low-latency/iterative computations (ML, graph algorithms).

B. In-Memory Computing (IMC)

- **Definition:** Middleware software to store/process data in RAM across computer cluster, in parallel.
- **Benefits:** RAM is much faster than disk. Native parallel processing.
- **Pillars:** RAM storage, parallel distributed processing.
- **Data Handling:** Relational DBs for structured, NoSQL for unstructured. Addresses volume and diversity.

XXVIII. Fast and Easy Big Data Processing with Spark

A. Spark Core

- **Programming Model:** General purpose, developers write applications composing arbitrary operators (mappers, reducers, joins, group-bys, filters).
- **Performance:** Caches working dataset in memory, avoids costly disk accesses.
- **Applications:** Low-latency, efficient iterative algorithms.

B. Spark Overview

- **Platform:** Cluster computing, fast and general purpose.
- **Speed:** Important for large datasets, extends MapReduce.
- **Generality:** Covers wide range of workloads (batch, iterative, interactive queries, streaming). Easy to combine processing types.
- **APIs:** Accessible in Java, Python, Scala, SQL.
- **Integrations:** Hadoop (HDFS), Kafka, AWS.

C. Spark Components (Unified Stack)

- **Spark Core:** Basic functionality: task scheduling, memory management, fault recovery, storage interaction. API for Resilient Distributed Datasets (RDDs).
- **Spark SQL:** For structured data. Querying via SQL/HQL. Intermixes SQL with RDD API in multiple languages.
- **Spark Streaming:** Processing live data streams (web server logs). API matches Spark Core RDD API. Fault tolerance, throughput, scalability.
- **MLlib:** Built-in ML functionality (classification, regression, clustering, collaborative filtering). Scalable across cluster.
- **GraphX:** For manipulating graphs (social networks) and graph-parallel computations. Extends RDD API.
- **Cluster Managers:** Scales from one to thousands of nodes. Runs over Hadoop YARN, Apache Mesos, Standalone Scheduler, Kubernetes.

XXIX. Apache Kafka

A. Messaging System Concepts

- **Purpose:** Transfer data between applications. Application focuses on core features, messaging system handles data transfer.
- **Types:**
Point-to-Point: Messages persisted in queue. One or more consumers, but only one consumes message. Message removed after read.
Pub-Sub: Messages persisted in topic. Many consumers can consume simultaneously. Producers (publishers), Consumers (subscribers).

B. Apache Kafka

- **Definition:** Distributed pub-sub and queuing messaging system. High throughput for distributed systems.
- **Benefits:** Reliability (distributed, partitioned, replicated, fault tolerant), scalability (add machines without downtime), durability (messages persisted on disk), performance (high throughput).
- **Fit:** Large scale data processing, offline and online message transfer. Integrates with Storm, Spark.

C. Kafka Architecture (Fundamentals)

- **Topics:** Stream of messages of a category. Stored in topics, split into partitions (immutable, ordered sequence, segment files). Minimum one partition per topic. Each message has unique "offset". Replicas are backups.
- **Brokers:** Simple system maintaining published data. Zero or more partitions per topic. Load balance. Stateless (use ZooKeeper for cluster state).
- **Producers:** Publishers of messages to topics. Sends data to brokers (appends to segment file/partition). Don't wait for acknowledgements.
- **Consumers:** Read data from brokers. Subscribe to topics, pull data. Maintain consumed messages via partition offset (notified by ZooKeeper). Can rewind/skip using offset.
- **Kafka Cluster:** More than one broker. Expandable without downtime. Manages persistence/replication.
- **Leader/Follower:** Leader handles reads/writes for partition. Follower copies leader's data. If leader fails, follower becomes new leader.
- **ZooKeeper:** Manages/coordinates Kafka brokers. Notifies producers/consumers about broker presence/failure.

D. Kafka Workflow

- **Pub-Sub:** Producers send to topic. Broker stores in partitions. Consumer subscribes, Kafka provides/saves offset to ZooKeeper. Consumer requests new messages. Kafka forwards. Consumer processes, acknowledges. Kafka updates offset in ZooKeeper.
- **Queue Messaging/Consumer Group:** Group of consumers with same "Group ID" subscribe. Messages shared among group members. Sharing continues until consumers = partitions.

E. Kafka CLI and Java APIs

- **CLI:** Server management (ZooKeeper, Kafka servers), Topic lifecycle (create, list, update, delete), Producer, Consumer.
- **Java APIs:**
Kafka Producer API: Producer class (connects to broker, closes connections), KafkaProducer methods (send, flush, metrics), ProducerRecord (key/value pair).
Kafka Consumer API: KafkaConsumer class (subscribe, unsubscribe, poll), ConsumerRecords (container for ConsumerRecord), ConsumerRecord (topic, partition, key/value).
Kafka Streaming API: For computations on messages.

XXX. Monitoring and Observability

A. Monitoring vs. Observability (Distinction)

- **Historical Monitoring:** Preserve of Operations engineers. Simple up/down checks. Evolved to more powerful tools.
- **Observability:** Rebranding, "devopsifying" monitoring. Captures what monitoring doesn't. Complementary. Not a novel idea (events, tracing, exception tracking are forms of observability).

B. Monitoring (Detailed)

- **Whitebox Monitoring:** Information from system internals (time series, logs, traces). Etsy's statsd was game changer.
- **Symptom-based Alerting:** Addresses "what's broken" (symptom) vs. "why" (cause).
- **Blackbox Monitoring:** From outside, answers "what's broken," alerts about ongoing problems.
- **Whitebox Monitoring:** For anticipated signals, known hard failure modes.
- **Monitorable Systems:** Design goal. Understand failure domain proactively. Identify small set of hard failure modes, core health metrics.
- **Actionable Monitoring Data:** Rules catch real incidents, simple/predictable. Data collection rarely exercised should be removed. Signals not in dashboards/alerts are candidates for removal. Provides bird's-eye view. Doesn't guarantee failure avoidance.

C. Observability at Twitter

- Superset of monitoring.
- **Four Pillars:** Monitoring, Alerting/visualization, Distributed systems tracing infrastructure, Log aggregation/analytics.

XXXI. Data Observability

A. Need for Data Observability

- No longer just for software. Critical for data teams due to data downtime, complex data stack.
- **DevOps Analogy:** DevOps integrates developers/IT for reliable software releases. Observability tracks/triage incidents to prevent downtime.

- **Observability Engineering Pillars:** Metrics (numeric over time), Logs (record of event), Traces (causally related events). Insights to predict behavior.
- **Data Downtime:** Common phenomenon (partial, erroneous, missing, inaccurate data). Erodes confidence.

B. Data Observability (Definition and Purpose)

- **Definition:** Comprehensive understanding of health/performance of data within systems.
- **Tools:** Automated monitoring, root cause analysis, data lineage, data health insights. Proactively detect, resolve, prevent anomalies.
- **Results:** Healthier pipelines, increased productivity, enhanced data management, higher customer satisfaction.

C. Five Pillars of Data Observability

1. **Freshness:** How up-to-date, cadence of updates. Critical for decision-making.
2. **Quality:** Looks at data itself (percent NULLS, uniques, accepted range). Can data be trusted?
3. **Volume:** Completeness of tables, health of sources. Detect sudden drops/increases.
4. **Schema:** Changes indicate broken data. Monitoring who/when changes occur.
5. **Lineage:** "Where" data broke. Upstream sources, downstream ingestors, teams, access. Collects metadata (governance, business, technical guidelines).

D. Importance and Key Features of Tools

- **Importance:** Data downtime wastes resources, erodes confidence.
- **Key Features:** Connects to existing stack seamlessly (no code modification). Monitors data at-rest. Minimal configuration (ML for anomaly detection). No prior mapping required. Provides rich context for triage/troubleshooting. Prevents issues by exposing data asset info.

E. Future of Data Observability

- Rapidly maturing, evolving space. Tremendous investor/customer interest.
- Heart of modern data stack (self-service analytics, collaboration, dbt unit tests, Airflow circuit breakers).
- Core component of data quality best practices (data mesh, data SLAs, data contracts).

XXXII. Machine Learning Observability

A. What is ML Observability?

- Tool to monitor, troubleshoot, explain ML models from research to production.
- Automatically surfaces issues, drills down to root cause, acts as guardrail.

B. Four Pillars of ML Observability

1. **Performance Analysis:** Fast actionable performance info on deployed models. Metrics: Accuracy, Recall, F-1, RMSE, Precision. Ensures performance hasn't degraded.
2. **Drift:** Monitors change in distribution over time (model inputs, outputs, actuals). Identifies stale models, data quality issues, adversarial inputs. Protects against degradation.
3. **Data Quality:** Checks identify hard failures in data pipelines between training/production. Monitoring cardinality shifts, missing data, type mismatch, out-of-range.
4. **Explainability:** Uncovers feature importance across environments. Introspects/understands why model made prediction. SHAP, LIME metrics.

C. Achieving ML Observability

- Deep understanding into model's data and performance across lifecycle. Doesn't just surface red/green light, but enables root cause/explanation for improvement.

XXXIII. Privacy

A. Definition and Challenges

- **Difficulty:** Notoriously hard to define.
- **Big Data Impact:** De-identified data (removed name, address) could be re-identified by combining datasets (birthday, zip code, gender).

B. Privacy Goals in ML

- **Assets to Protect:** Identity of data contributors, raw dataset, features, model itself, its input.
- **Identity Privacy:** Ensuring anonymity of contributors from model creators/owners and adversaries.

- **Raw Dataset Privacy:** Sensitive info (X-rays, energy consumption/location). Compromising can expose personal details. Store encrypted.
- **Feature Datasets Privacy:** As important as raw dataset. Compromising training/validation sets can reveal raw data.
- **Model Privacy:** Secrecy of model and parameters. Exposing functionality (responses to queries) can reveal internal structure.
- **Input Privacy:** Input to model may be sensitive. Only output revealed. Requires keeping input secret, limiting info disclosed through response.

C. Two Key Ideas (Theoretical)

- **k-anonymity:** In a dataset, for any combo of categories, there are at least 'k' individuals. Achieved by generalizing/suppressing data (e.g., month/year for birthday, first digits of zip code).
- **Differential Privacy:** Mathematical method adding noise to data. Probabilistic guarantees that inferences about specific individual cannot be made from noisified data. Outcome not significantly dependent on individual's record. Achieved by random noise mechanisms (Laplace).

D. Threat Models and Attacks

- **Adversary Models: White Box:** Knows model structure, parameters, possibly part of raw dataset. Can interact with model.
 - **Black Box:** No knowledge of model/parameters, but can query model/observe responses.
1. **Types of Attacks: Membership Inference Attacks (Black Box):** Determine if data point was part of raw/training dataset. Exploits different model behavior on seen vs. unseen data. Compromises raw/feature dataset privacy.
 2. **De-anonymisation Attacks (White Box):** Identify individual from anonymized dataset (by linking with other info). Compromises identity privacy.
 3. **Reconstruction Attacks (White Box):** Construct raw dataset by reverse engineering feature datasets. Compromises raw dataset privacy.
 4. **Model Extraction Attacks (Black Box):** Construct approximation of model by collecting query/response pairs. Violates model privacy.
 5. **Model Inversion Attacks (Limited Black Box):** Infer properties of training dataset, even if not explicitly stored. Can follow model extraction.

E. Methods to Preserve Privacy

- **Privacy-enhancing Measures:** Technical and Institutional.
- **Technical Measures:**
 - Access Controls:** "Need to know" basis for sensitive data. Periodically revisit access.
 - Access Logging:** Track who/when accesses data. Understand patterns, detect inappropriate access, preserve evidence. Can inform schema refactoring to minimize exposure.
- **Data Minimization:** Collect/use only necessary data.
- **Data Separation:** Separate legitimate business data from sensitive data (names, addresses) if not relevant to ML model.
- **Institutional Measures:**
 - Ethics Training:** For all involved in ML product design (general ethics, domain-specific).
 - Data Access Guidelines:** Explicit rules for appropriate data access/use, prohibited uses. Institutional culture with accountability.
- **Privacy by Design:** Privacy intrinsic to design from the start, addressed at all stages. Holistic approach.

XXXIV. Privacy-Preserving Techniques (Practical)

A. Overview

- Techniques to strengthen ML algorithms to protect individual data.

B. Specific Techniques

- **Anonymization:**
 - Goal:** Protect individual privacy in dataset. Remove direct identifiers.
- **Additional Steps:** k-anonymity (information indistinguishable from k-1 others), l-diversity (k-anonymity plus diversity for sensitive attributes).
- **Challenge:** Providing privacy while preserving utility.
- **Differential Privacy (Revisited):**
 - Goal:** Reveal useful info about dataset without private info about individual.
- **Guarantee:** Attacker cannot link record to individual based on differentially-private output. Outcome not significantly dependent on individual's participation.

- **Achievement:** Adding random noise to result via mechanisms (Laplace, exponential, randomized response).
- **Homomorphic Encryption: Cryptographic Method:** Compute on encrypted data; decrypted output is identical to computation on unencrypted input.
- **Process:** Data owner encrypts, shares with third party. Third party computes on encrypted data, returns encrypted result. Owner decrypts. Third party never sees unencrypted data.
- **Limitations:** Limited to specific computations (addition, multiplication over integers). Noise added during encryption accumulates, affecting number of computations possible.
- **Multi-party Computation (MPC): Technology:** Multiple parties compute function on private inputs without revealing them. Parties are independent, don't trust each other.
- **Guarantee:** Participants learn nothing more than what they can learn from output and their own input.
- **Trade-offs:** High communication/computation costs.
- **Federated Learning: Decentralized ML:** Limits exposed information from contributors' datasets. Controls risk of compromising dataset/identity privacy.
- **Process:** Central model distributed to n participants. Each participant trains locally on own dataset, produces local parameter update. Participants send updates to central authority. Central authority aggregates updates, updates central model. Process continues.
- **Benefits:** Privacy of raw/feature datasets, identity privacy. No vast centralized data collection.
- **Challenges:** Communication overhead, scalability concerns due to device availability.

Quiz: Data Management for ML

Instructions: Answer each question in 2-3 sentences.

1. Describe the primary purpose of the data management stage in modern ML environments.
2. Explain the key difference between structured and unstructured data, and provide an example of where each might be stored.
3. What is data sampling in the ingestion phase, and what trade-offs does it involve?
4. Why is data validation considered a crucial operation during the processing phase of data management?

5. Briefly explain the concept of data as a "liability" in ML systems, citing one specific concern.
6. Differentiate between the terms "row-major" and "column-major" formats, and state when each is generally preferred.
7. What is the main advantage of using a declarative query language like SQL compared to imperative code for data querying?
8. Describe "schema evolution" in the context of data flow through databases and why it is important.
9. Identify two key benefits of using a message broker in an asynchronous message-passing system.
10. What is the fundamental difference between data warehousing and data lakes in terms of data structure and purpose?

Quiz Answer Key

1. The primary purpose of the data management stage is to transform raw data into a format and storage organization suitable for later ML processes. This includes preparing data for model training, anonymizing sensitive items, and ensuring data is ready for operations like deletion or future exploration.
2. Structured data is quantitative with a predefined schema, highly organized, and stored in tabular formats; it can be easily processed by simple code. Unstructured data is qualitative, lacks a standard schema, and cannot be processed by conventional tools. Structured data is typically stored in data warehouses, while unstructured data is found in data lakes.
3. Data sampling in the ingestion phase involves processing only a subset of available data to save computational costs, as ML training can be expensive. The trade-off is that any reduction in data may negatively impact the quality or accuracy of the ML model, so the quality cost must be weighed against the savings.
4. Data validation is crucial because efficient ML models cannot perform well with bad data; it ensures the accuracy and quality of incoming data. It helps detect errors, especially from external sources or bugs in data collection code, which is vital for maintaining model integrity in production.
5. Data as a liability means that its acquisition, collection, and curation can expose unexpected complexities and potential harm. For example, data must be collected and stored in compliance with applicable laws (like GDPR regarding PII), requiring explicit

consent and the ability to delete data upon request, failure of which can lead to legal and reputational risks.

6. Row-major formats (like CSV) store consecutive elements in a row next to each other in memory, making row access and data writes faster. Column-major formats (like Parquet) store consecutive elements in a column contiguously, allowing for faster column-based reads, especially beneficial for large datasets with many features.
7. The main advantage of a declarative query language like SQL is that it allows users to specify *what* data they need (the pattern, conditions, and transformations) without instructing *how* to achieve that goal. This empowers the database system's query optimizer to decide the most efficient execution plan, including parallel execution, often improving performance.
8. Schema evolution refers to the ability to modify a database schema over time (e.g., adding a new column) without necessarily rewriting all existing data. This is important for databases because data often outlives code, allowing for system updates and compatibility with both older and newer application versions concurrently.
9. Two key benefits of using a message broker in an asynchronous message-passing system are improved system reliability and logical decoupling. Message brokers can act as a buffer if the recipient is unavailable or overloaded, and they allow the sender to publish messages without needing to know the recipient's specific IP address or port number.
10. Data warehousing stores highly formatted, structured data with a predefined schema, optimized for specific analytical and reporting use cases like business intelligence dashboards. In contrast, data lakes store raw, unstructured, and semi-structured data without an imposed schema, serving as a flexible sandbox for innovative data projects and data scientists.

Glossary of Key Terms

- **Data Management:** The process of transforming raw data into a suitable format and organizational structure for use in subsequent stages of the machine learning pipeline, including training, anonymization, and deletion.
- **Structured Data:** Data that is quantitative, has a predefined data model/schema, is highly organized, and stored in tabular formats (e.g., spreadsheets, relational databases).
- **Unstructured Data:** Qualitative data without a standard data model/schema that cannot be processed and analyzed using conventional methods (e.g., text, images, audio).

- **Semi-structured Data:** Data that lacks a specific data model/schema but includes tags and semantic markers, lying between structured and unstructured data (e.g., JSON, XML).
- **Dataset Augmentation:** Techniques used to expand a small dataset into a larger one, often by generating synthetic data or applying transformations, to improve model training.
- **Data Ingestion:** The phase where data is received into a system and written to storage for further processing, often involving filtering, selection, and sampling.
- **Data Validation:** The process of checking the accuracy and quality of incoming data, especially from external sources or against a predefined schema, to ensure it is suitable for ML model training.
- **Data Normalization:** A set of techniques used to transform input data into a similar scale, ensuring consistency and preventing features with larger values from dominating.
- **Data Enrichment:** The process of combining existing data with data from other sources to add more context or features to the dataset.
- **Data Labeling:** The process of identifying a given event or record by bringing in confirmation from an outside source (often human) to assign a specific category or value, crucial for supervised ML.
- **Metadata:** Data that provides information about other data, such as attributes, definitions, versions, and usage restrictions, essential for understanding and managing datasets and models.
- **Data Architecture:** The design of systems to support an enterprise's evolving data needs, achieved through flexible and reversible decisions based on evaluating trade-offs.
- **Data Warehouse (DW):** A central data hub designed for reporting and analysis, typically storing highly formatted and structured data with a predefined schema.
- **Data Lake (DL):** A repository that stores raw data of any type (structured, unstructured, semi-structured) from any system, without imposing a tight structural limitation.
- **Data Lakehouse:** An architecture that combines the data management features (like ACID transactions) of a data warehouse with the low-cost storage of a data lake, aiming for convergence.
- **Modern Data Stack (MDS):** A suite of cloud-based, plug-and-play tools designed for data integration, transformation, and analysis, emphasizing modularity, ease of use, and cost-effectiveness.

- **Lambda Architecture:** A big data processing architecture that combines batch processing for historical accuracy and real-time (speed layer) processing for low-latency insights, with a serving layer to combine results.
- **Kappa Architecture:** An alternative to Lambda architecture that uses a single stream processing platform as the backbone for all data handling, facilitating both real-time and batch processing on the same data.
- **Dataflow Model:** A unified approach to batch and stream processing that views all data as events and performs aggregations over various types of windows, aiming to use nearly identical code for both.
- **Declarative Query Language:** A language (like SQL) where the user specifies *what* data they want to retrieve or manipulate, rather than *how* to perform the operations.
- **Imperative Code:** Programming code that explicitly specifies *how* a task should be performed by providing a sequence of instructions in a particular order.
- **MapReduce:** A programming model for processing large amounts of data in bulk across many machines, using "map" functions to process data in parallel and "reduce" functions to aggregate results.
- **Data Serialization:** The process of converting a data structure or object state into a format that can be stored or transmitted and reconstructed later (e.g., JSON, Parquet).
- **Schema Evolution:** The ability of a database system to adapt to changes in the data schema over time (e.g., adding new columns) without requiring a full rewrite of existing data.
- **Remote Procedure Call (RPC):** A protocol that allows a program to cause a procedure (subroutine) to execute in a different address space (typically on another computer on a shared network) as if it were a local procedure call.
- **Message Broker:** An intermediary system in asynchronous message-passing that stores messages temporarily, buffering requests and decoupling senders from recipients.
- **Data Pipeline:** A series of steps that allow data to flow from one system to another, typically involving pulling data from a source, applying transformations, and pushing it to a destination for analytics, ML, or AI.
- **DataOps:** An emerging set of practices, processes, and technologies for building and enhancing data and analytics pipelines to meet business needs quickly, integrating concepts from Agile, Lean, and DevOps.

- **Feature Engineering:** The process of selecting and transforming variables (features) from raw data to create new, more informative features that improve the performance of a predictive model.
- **Feature Store:** A storage system designed to store extracted feature values so they can be quickly and consistently read during model training and inference, especially useful for sharing features among multiple models.
- **Model Registry:** A centralized repository for storing, versioning, managing, and documenting machine learning model artifacts and metadata, bridging the gap between experimentation and production.
- **MLOps (Machine Learning Operations):** An ML engineering culture and practice that aims to unify ML system development (Dev) and ML system operation (Ops), advocating for automation and monitoring at all steps of the ML system lifecycle.
- **Continuous Training (CT):** A new property unique to ML systems, concerned with automatically retraining and serving models in response to new data or performance degradation.
- **Hadoop:** An open-source framework that provides massive data storage (HDFS) and faster data processing (MapReduce) for large datasets across clusters of commodity hardware.
- **In-Memory Computing (IMC):** A middleware software that allows data to be stored and processed in RAM across a cluster of computers, enabling faster computation speeds by avoiding disk I/O.
- **Apache Spark:** A fast and general-purpose cluster computing platform that extends the MapReduce model, efficiently supporting diverse workloads including batch, iterative algorithms, interactive queries, and streaming.
- **Apache Kafka:** A distributed pub-sub and queuing messaging system designed for high throughput, reliability, and fault tolerance, used for both offline and online message transfer in large-scale data processing.
- **Monitoring:** The traditional practice of observing and tracking the health and performance of systems, often focusing on known failure modes and symptom-based alerting.
- **Observability:** A more recent concept that goes beyond monitoring, aiming to provide highly granular insights and rich context into the internal behavior of systems for debugging unknown failure modes.

- **Data Observability:** An organization's comprehensive understanding of the health and performance of the data within their systems, employing automated monitoring, root cause analysis, and data lineage to detect, resolve, and prevent data anomalies.
- **ML Observability:** A tool or practice used to monitor, troubleshoot, and explain machine learning models as they move from research to production, focusing on performance analysis, drift, data quality, and explainability.
- **k-anonymity:** A privacy technique ensuring that in a given dataset, for any combination of identifying attributes, there are at least 'k' individuals who share those attributes, making re-identification difficult.
- **Differential Privacy:** A mathematical method that adds noise to data to provide probabilistic guarantees that inferences about a specific individual cannot be made from aggregate reporting, even if the individual's data is part of the dataset.
- **Homomorphic Encryption:** A cryptographic method that allows computations to be performed directly on encrypted data, yielding an encrypted result that, when decrypted, is identical to the result of the computation on the original unencrypted data.
- **Multi-party Computation (MPC):** A technology that enables multiple parties to jointly compute a function on their private inputs without revealing those inputs to each other.
- **Federated Learning:** A decentralized machine learning approach where a central model is trained on new private datasets by having each data contributor perform local training and then sending only model updates (not raw data) back to the central authority.